

{ OpenSpec: Spec-Driven Development nell'era degli Agenti AI }

Un nuovo paradigma per la collaborazione tra Umani e Intelligenza Artificiale

Speaker: Matteo Baccan

Evento: DevFest Modena 2026 — 3/4 Ottobre 2026, Modena

Track: AI & Machine Intelligence

{ Dal Vibe Coding a OpenSpec }

Il Problema: Agenti AI senza struttura

Gli agenti AI sono collaboratori attivi, ma senza struttura generano **vibe coding**: prompt informali, contesto volatile, drift dei requisiti.

- I requisiti vivono solo nella chat: nessuna documentazione persistente
- Allucinazioni architetturali e scelte inconsistenti
- Debito tecnico invisibile e difficilmente tracciabile

La Soluzione: Spec-Driven Development

Lo **Spec-Driven Development** inverte il paradigma: **la struttura prima del codice**.

	Vibe Coding	Spec-Driven Development
Flusso	Prompt → Codice	Idea → Specifica → Architettura → Task → Codice
Artefatti	Nessuno (solo chat)	<code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs
Verificabilità	Impossibile	Ogni fase produce output controllabile

Cos'è OpenSpec?

OpenSpec è un framework open-source progettato per la Spec-Driven Development.

Non richiede paywall o API key proprietarie per esistere come metodo, non è vincolato a un IDE specifico o a un singolo vendor AI.

È un sistema basato su file Markdown che funge da "controllo di versione per l'intento"

- **zero lock-in**
- specifiche portabili
- valore che resta nel repository.

OpenSpec: i tre pilastri

Pilastro 1: Brownfield-First ($1 \rightarrow n$)

Ottimizzato per l'evoluzione di codebase esistenti, non solo per prototipi greenfield ($0 \rightarrow 1$).

Permette di modificare logiche complesse in sicurezza, senza riscrivere tutto da zero.

Pilastro 2: Architettura Leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: Agnosticismo Totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.

La Filosofia di OpenSpec

L'idea centrale è che la specifica sia la "fonte della verità", non il codice.

I documenti fungono da istruzioni eseguibili e vincolanti per gli agenti AI, non solo come suggerimenti o linee guida.

Il risultato non nasce da un'interpretazione libera: con OpenSpec c'è **solo intento deterministico**, espresso in un contratto scritto che l'agente deve seguire.

Living Documentation

Le specifiche sono mantenute in Git accanto al codice sorgente.

Non diventano mai obsolete perché si evolvono in parallelo al sistema, rendendo la documentazione un prodotto vivo dello sviluppo.

OpenSpec vs Strumenti di Project Management

I ticket nei sistemi tradizionali (come Jira o Linear) sono ottimi per gli umani, ma difficili da consultare in tempo reale dagli agenti AI.

OpenSpec porta le specifiche direttamente nel repository, dove l'agente opera.

Indipendenza dagli Strumenti AI

OpenSpec è universale. Supporta oltre 30 strumenti e ambienti di sviluppo diversi senza legarsi a un ecosistema proprietario.

Se un team decide di cambiare modello AI, estensione IDE o piattaforma, processi e specifiche rimangono intatti e validi.

Le specifiche vivono in Git: il valore resta tuo, non del vendor.

Ecosistema agnostico: zero lock-in come scelta

Nodo centrale	Ecosistema collegabile
OpenSpec come livello stabile di intento e governance nel repository	Kilo Code, Cursor, Claude Code, Windsurf, GitHub Copilot e altri agenti compatibili

Capacità	Impatto operativo
Integrazioni intensive	Uso tramite CLI e slash commands nativi nei principali IDE/LLM, senza riscrivere le specifiche
Supporto universale (AGENTS.md)	Le istruzioni restano leggibili da assistenti diversi (anche futuri), preservando storico e coerenza del processo

Risultato: cambi strumento quando vuoi, senza perdere memoria progettuale né controllo sull'intento.

Approccio "Brownfield-First"

Molti framework AI sono ottimizzati per progetti nuovi (greenfield, $0 \rightarrow 1$).

OpenSpec brilla nei progetti esistenti (brownfield, $1 \rightarrow n$), dove l'integrazione di nuove funzionalità senza rompere le vecchie è cruciale.

{ La Meccanica di OpenSpec }

L'Architettura del Framework

Tutto risiede in una singola directory radice all'interno del progetto:

```
openspec
```

Questa directory agisce come la memoria a lungo termine dell'agente AI.

La Separazione dello Stato

Il framework separa fisicamente:

1. La descrizione dello stato attuale del sistema.
2. Le proposte per le modifiche future.

Questa distinzione garantisce transizioni sicure e modifiche parallele.

L'architettura a due cartelle: specs/

L'Essere — la fonte della verità.

Documentazione del comportamento attuale del sistema, organizzata per domini logici. L'agente la consulta prima di proporre o scrivere codice.

- `specs/auth/spec.md`
- `specs/payments/spec.md`
- `specs/ui/spec.md`

L'architettura a due cartelle: **changes/**

Il Divenire — il workspace di modifica.

Laboratorio isolato per ogni nuova feature o bug fix. Nessun conflitto con le specifiche principali finché non si archivia.

- **changes/add-oauth-login/**
 - **proposal.md**
 - **design.md**
 - **tasks.md**
 - **specs/...** (Delta)

Il Cuore: La Directory specs/

Contiene la documentazione consolidata del comportamento attuale del software.

È la "**Source of Truth**" a cui l'agente deve attenersi prima di proporre o scrivere codice.

Organizzazione per Domini

I file in `specs/` sono organizzati per domini logici (es. `auth/`, `payments/`, `ui/`).

Ogni cartella ospita un file `spec.md` che descrive esattamente le capacità di quel comparto.

Il Workspace: La Directory **changes/**

Ospita le proposte di modifica. Ogni nuova feature o bug fix ottiene una propria cartella isolata.

Questo permette al team di preparare modifiche architetturali in modo pulito e strutturato.

Isolamento delle Modifiche

Lavorare in `changes/` previene i conflitti.

Più sviluppatori (e più agenti AI) possono preparare funzionalità diverse simultaneamente, senza inquinare la logica base finché non sono pronti.

Configurazione: Il File `config.yaml`

Funziona come la "costituzione" tecnica del progetto.

Sostituisce configurazioni frammentate fornendo un set di regole chiare, stack tecnologici e convenzioni di sviluppo.

Iniezione Attiva del Contesto

A differenza di un normale `README`, il contenuto di `config.yaml` viene **iniettato attivamente** nella finestra di contesto dell'agente in ogni interazione di pianificazione.

Garantisce che l'agente non dimentichi mai lo stack del team.

Governance attiva: come `config.yaml` guida ogni richiesta

<code>config.yaml</code>	Funzione	Effetto sull'agente
<code>schema</code>	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
<code>context</code>	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
<code>rules</code>	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.

Gli Artefatti della Pianificazione

Una cartella di modifica genera sempre un set standard di artefatti Markdown:

- `proposal.md`
- `design.md`
- `tasks.md`
- Le Delta Specs

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	proposal.md	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	design.md	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	tasks.md	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni ADDED , MODIFIED , REMOVED .

Flusso completo: **proposal.md** → **design.md** → **tasks.md** → Delta → consolidamento → **specs/**

L'Artefatto 1: `proposal.md`

Cattura l'intento strategico. Risponde alle domande "perché stiamo facendo questa modifica?" e "qual è lo scopo principale?".

È il punto di allineamento iniziale tra umano e macchina.

Why

Gli utenti abbandonano la registrazione: serve il login social.

What Changes

- Aggiunta autenticazione OAuth (Google) al flusso di login
- Nessun impatto sul login email/password esistente

L'Artefatto 2: design.md

È l'ancora tecnica. Delinea scelte di database, flussi di dati e specifiche librerie da utilizzare.

Impedisce all'agente di deviare dai pattern stabiliti "immaginando" soluzioni creative ma errate.

Decisioni tecniche

- Libreria: passport con strategy passport-google-oauth20
- Nel DB solo l'id utente Google: nessun token persistito
- Sessioni: riuso del meccanismo esistente (Redis)

L'Artefatto 3: `tasks.md`

Una checklist numerata di azioni da compiere.

Funge da registro di avanzamento. L'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

1. Backend

- [x] 1.1 Aggiungere endpoint GET /auth/google
- [x] 1.2 Gestire il callback e creare l'utente
- [] 1.3 Test di integrazione del flusso completo

L'Importanza dei Task Atomici

I task in `tasks.md` devono essere sufficientemente piccoli da essere implementati dall'agente in autonomia, senza richiedere costanti chiarimenti o interventi dell'utente.

L'Artefatto 4: Delta Specs

La vera innovazione di OpenSpec. Invece di riscrivere l'intera specifica di sistema, l'agente crea una specifica "differenziale" (Delta) che mostra solo cosa cambierà.

Perché usare le Delta Specs?

In codebase enormi, rigenerare tutta la documentazione consumerebbe troppi token e tempo.

Le Delta Specs si focalizzano solo sull'incremento funzionale, ottimizzando i costi e mantenendo alta l'accuratezza.

Anatomia di una Delta Spec

Le Delta Specs utilizzano intestazioni chiare per istruire il processo di fusione futuro:

1. **ADDED Requirements**
2. **MODIFIED Requirements**
3. **REMOVED Requirements**

Delta Tags: ADDED

Definisce comportamenti completamente nuovi.

Esempio: L'aggiunta di un sistema di autenticazione a due fattori in un progetto che prima aveva solo login base.

```
## ADDED Requirements
```

```
### Requirement: Two-Factor Authentication
```

```
The system SHALL richiedere un secondo fattore al login.
```

```
#### Scenario: Login con 2FA attiva
```

- WHEN un utente con 2FA invia credenziali valide
- THEN viene richiesto un codice OTP prima di creare la sessione

Delta Tags: MODIFIED

Descrive l'alterazione di comportamenti esistenti.

Richiede di riscrivere l'intero requisito aggiornato, garantendo che nessuna regola pregressa venga persa per distrazione dell'agente.

```
## MODIFIED Requirements
```

```
### Requirement: Session Duration
```

```
The system SHALL scadere la sessione dopo 30 minuti di  
inattività (prima: 24 ore), mantenendo il "ricordami"  
per i dispositivi fidati.
```

Delta Tags: REMOVED

Segnala funzionalità deprecate o da rimuovere.

Garantisce che il codice morto venga eliminato e che i vecchi test associati vengano disattivati correttamente.

```
## REMOVED Requirements
```

```
### Requirement: Legacy XML Export
```

```
**Reason**: sostituito dall'export JSON
```

```
**Migration**: i client devono passare a /api/export/json
```

{ Il Linguaggio dell'Intento }

Scrivere Specifiche Efficaci

Le specifiche non devono essere istruzioni di programmazione step-by-step. Devono descrivere il comportamento osservabile del sistema dall'esterno. Sono contratti di business, non tutorial di codice.

Il linguaggio dell'intento: EARS + BDD

Asse	EARS (obbligazione)	BDD (esecuzione)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave normative: SHALL , MUST , SHOULD	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con EARS, poi ne proviamo l'esecuzione con BDD.

Sintassi EARS

OpenSpec adotta l'approccio EARS (Easy Approach to Requirements Syntax).
L'obiettivo è minimizzare l'ambiguità utilizzando parole chiave vincolanti per definire i livelli di obbligazione.

Le parole chiave: SHALL e MUST

Rappresentano comportamenti obbligatori e non negoziabili.

Se un test fallisce su un requisito **SHALL**, l'agente sa che l'implementazione è categoricamente errata.

La parola chiave: **SHOULD**

Indica una raccomandazione o un comportamento desiderato, ma con margini di flessibilità.

Permette all'agente di adattarsi a vincoli tecnici imprevisti durante la scrittura del codice.

Scenari Verificabili

Ogni requisito deve essere accompagnato da scenari pratici derivati dal Behavior-Driven Development (BDD).

Forniscono all'agente esempi inequivocabili di successo e fallimento.

Il Modello GIVEN / WHEN / THEN

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").

Dalla Specifica ai Test

Con una struttura GIVEN/WHEN/THEN chiara, l'agente AI è in grado di generare automaticamente i test unitari o e2e corrispondenti, chiudendo il ciclo della qualità del software.

{ Il Ciclo di Esecuzione }

Il Ciclo Operativo (Workflow)

OpenSpec definisce una macchina a stati immutabile a tre fasi per ogni modifica:

1. Propose — `/opsx:propose`
2. Apply — `/opsx:apply`
3. Archive — `/opsx:archive`

Ogni transizione ha un output verificabile e impedisce di passare alla fase successiva senza allineamento.

Fase zero opzionale: `/opsx:explore` — un "thinking partner" senza vincoli che legge il codice e pesa le alternative prima di formalizzare la proposta.

Il profilo esteso aggiunge comandi come `/opsx:verify`, `/opsx:ff`, `/opsx:continue` e `/opsx:onboard`.

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo deterministico (Apply), consolida nella Source of Truth (Archive).

Fase 1: Proposta (Propose)

L'agente non scrive codice. Analizza la richiesta dell'umano e genera la cartella in **changes/** con la proposta, i task e le Delta Specs.

La Revisione dell'Intento

Questo è il momento chiave per lo sviluppatore umano.

Modificare un file Markdown errato richiede pochi secondi. Correggere un'architettura software errata dopo che è stata scritta richiede giorni.

La Scissione Cognitiva dell'Agente

Perché separare proposta e implementazione aumenta la qualità del codice generato?

Nel vibe coding il modello divide il proprio **budget di attenzione** tra tre compiti contemporaneamente:

1. Risolvere il problema di business
2. Progettare l'architettura software
3. Scrivere codice sintatticamente corretto

Risultato: precisione ridotta su tutti e tre i fronti, più iterazioni, più errori architetturali.

Focalizzazione: il 100% su un compito alla volta

Con OpenSpec ogni fase riceve l'intera capacità computazionale del modello:

Fase	Focus dell'agente
Propose	100% sull'architettura e le decisioni di design
Apply	100% sulla correttezza e completezza del codice
Archive	100% sulla coerenza documentale e la Source of Truth

Più focus su un compito → Meno errori → Meno cicli di correzione

Fase 2: Implementazione (Apply)

Solo quando gli artefatti sono approvati, l'agente inizia l'implementazione vera e propria.

Legge i compiti in `tasks.md` e produce il codice rigorosamente entro i confini stabiliti nel `design.md`.

L'Esecuzione dell'Agente

Durante l'Apply, l'agente AI si trasforma in un mero esecutore.
Smette di indovinare le intenzioni e si limita a tradurre specifiche perfette in codice funzionante.

Gestione della Qualità

Prima di considerare chiusa la fase di implementazione, l'agente verifica il codice prodotto rispetto alle specifiche stabilite, correggendo autonomamente eventuali deviazioni dai requisiti MUST/SHALL.

Fase 3: Archiviazione (Archive)

Una volta che il codice funziona ed è stato testato, la modifica viene conclusa. Questo è il momento in cui l'intento temporaneo diventa documentazione permanente.

La Fusione della Verità

Il sistema unisce automaticamente le Delta Specs nei file originali della cartella **specs/**.

Il sistema si aggiorna: la nuova funzionalità fa ora ufficialmente parte della "Source of Truth" del progetto.

L'Archivio Storico

La cartella originale della modifica viene spostata in un archivio storico (es. ordinato per data).

Si preserva così l'audit trail delle decisioni architetturali per i futuri sviluppatori.

Funzionalità Avanzate: Schemi Personalizzati

Team diversi hanno processi diversi. OpenSpec permette di definire schemi personalizzati per generare artefatti specifici (es. audit di sicurezza obbligatori) prima dell'implementazione.

Sviluppo in Parallelo

In team numerosi, la SDD scala brillantemente.

La separazione in cartelle di modifica (**changes/**) consente di gestire backlog complessi senza corrompere la specifica di base del sistema.

Sviluppo Parallelo su Branch

OpenSpec è compatibile con qualsiasi strumento che supporti branch isolati. Team e agenti possono lavorare su specifiche diverse in parallelo, su branch separati, e integrare le modifiche nella Source of Truth principale solo quando pronte.

Parallelismo e sincronizzazione

OpenSpec scala su tre assi:

Orizzontale — branch paralleli

Più agenti su feature diverse, ognuno con la propria cartella **changes/**; il ciclo **Propose → Apply → Archive** guida il merge ordinato nella Source of Truth.

Verticale — Project Management

Via MCP dialoga con Linear/Jira: il backlog resta allineato allo stato reale del codice.

Cross-repo — Stores (beta)

La pianificazione vive in un repository dedicato e condiviso; i repo di codice si allineano dopo.

Messaggio chiave: OpenSpec scala in orizzontale, in verticale e tra repository

Sincronizzazione con il Project Management

A livello enterprise, OpenSpec può dialogare tramite protocolli MCP con strumenti di ticket come Linear o Jira.

Il backlog aziendale resta allineato con lo stato reale del codice, riducendo il lavoro amministrativo e i disallineamenti tra piano e implementazione.

Orchestrazione Multi-Agente

OpenSpec non è solo un'interfaccia tra umano e LLM.

È un livello di astrazione che permette a **team di agenti specializzati** di collaborare autonomamente su sistemi complessi.

Invece di affidarsi a un singolo agente generico, il supervisore umano coordina ruoli distinti.

I Ruoli nel Sistema Multi-Agente

Agente Architetto

Consuma `config.yaml` e produce Record di Decisione Architetture (ADR).

Genera la struttura iniziale OpenSpec e pianifica la suddivisione logica del sistema.

Agente Orchestratore

Legge il `tasks.md` prodotto dall'architetto e delega il lavoro ai team specializzati.

Supervisiona l'avanzamento senza scrivere codice direttamente.

Team di Sviluppo AI

Agenti iperspecializzati per dominio: Data Access, Business Logic, Frontend.

Lavorano in parallelo sui propri task atomici, vincolati alle specifiche.

{ Dal Team al Legacy }

Il Problema del Codice Legacy

Come si introduce OpenSpec in un progetto esistente di 100.000 righe di codice senza documentazione?

Non riscrivendo tutto da zero.

Brownfield adozione incrementale

La scala dell'adozione	<code>/opsx:onboard</code> (reverse engineering)
1. Non fermare il mondo Nessun bisogno di produrre specifiche upfront per milioni di righe.	Comando dedicato per accelerare la partenza su moduli complessi o poco documentati.
2. Sviluppo just-in-time Usa OpenSpec sulla prossima feature o sul bug critico, dove stai già intervenendo.	L'agente analizza il codice esistente per estrarre le regole di business già implementate.
3. Accumulo organico La Source of Truth cresce naturalmente ad ogni ciclo Propose -> Apply -> Archive .	Generazione assistita di file spec.md retroattivi, da rifinire e validare con il team.

Principio guida

Non serve documentare tutto prima di iniziare.

OpenSpec non richiede un Big Bang documentale: non si blocca lo sviluppo per scrivere mesi di specifiche retroattive.

Il punto di ingresso è il lavoro che si sta già facendo: la prossima feature, il prossimo bug critico.

- Ogni intervento sul codice diventa un'occasione per aggiungere una specifica.
- La Source of Truth cresce ciclo dopo ciclo, in modo organico e sostenibile.
- `/opsx:onboard` può accelerare il processo su moduli complessi o poco documentati, ma è uno strumento opzionale — non un prerequisito per partire.

Adozione Incrementale

La strategia suggerita è documentare solo ciò che si tocca.

Se si deve modificare il modulo di pagamento, si documenta solo quello. Nel tempo, la documentazione crescerà organicamente.

Reverse Engineering delle Specifiche

Il comando `/opsx:onboard` analizza il codice sorgente esistente per generare architetture OpenSpec di base tramite AI, accelerando l'adozione su progetti già avviati.

Memoria Episodica per il Codice Legacy

In un contesto brownfield un singolo prompt iniziale non basta.

OpenSpec supporta un'architettura a **memoria episodica** che trasforma i fallimenti in istruzioni correttive persistenti:

Componente	Funzione
Agente Riflettore	Si attiva ogni N scambi, analizza la cronologia recente, identifica errori architetturali e produce una root-cause analysis
Agente Curatore	Legge l'analisi del Riflettore e la traduce in mutazioni di regole (aggiunte, aggiornamenti, rimozioni)
Playbook Eseguitibile	Regole salvate in un file JSON di sessione: l'agente costruisce il proprio manuale tattico mentre lavora

L'agente impara dai propri errori e non li ripete nella stessa sessione.

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare **Archive** e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in **design.md**.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare **ADDED**, **MODIFIED**, **REMOVED** in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.

Anti-Pattern 1: Debito Speculativo

Il più grave errore è dimenticarsi di completare la fase di **Archive**.

Creare modifiche che rimangono perennemente in sospeso distrugge l'affidabilità della "Source of Truth", riportando il team al caos.

Anti-Pattern 2: Micro-management dell'AI

Scrivere dettagli implementativi nelle specifiche principali significa trasformare i requisiti in pseudo-codice.

Le specifiche devono descrivere il comportamento; il "come" va spostato in **design.md**, lasciando all'agente solo esecuzione e verifica.

Anti-Pattern 3: Burocrazia per modifiche banali

Usare una SDD completa per un cambiamento minimo e non rischioso introduce attrito inutile.

Serve progressive rigor: più la modifica è piccola, più il processo deve restare leggero e orientato alla chiarezza.

Anti-Pattern 4: Ignorare la formattazione Delta

Saltare i tag **ADDED**, **MODIFIED**, **REMOVED** rompe il meccanismo di consolidamento e rende i merge meno affidabili.

Le Delta Specs non sono un vezzo sintattico: sono il contratto strutturale che permette fusioni pulite nella Source of Truth.

{ Perché Tutto Questo Conta }

Impatto strategico

Determinismo + intento persistente = scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
Specifiche eseguibili: più accuratezza al primo tentativo, meno rilavorazioni.	La conoscenza architetturale non muore nella chat: resta al team e resiste al turnover.	Il valore umano passa dallo "scrivere tutto" al garantire che il codice sia quello necessario.

Sintesi: quando l'implementazione è deterministica e l'intento rimane persistente, il sistema scala senza perdere qualità.

L'Impatto Strategico

La SDD trasforma l'incertezza dei prompt in un processo ingegneristico prevedibile.

- Accuratezza al primo tentativo elevatissima.
- Protezione del know-how aziendale.
- Scalabilità del team.

Il Nuovo Professionista: l'Agentic Engineer

Lo SDD forma una nuova categoria professionale che va oltre il programmatore tradizionale.

Un **ingegnere di sistemi agentici** che padroneggia:

- La progettazione del contesto per gli agenti
- La segmentazione di carichi di lavoro complessi in task atomici
- La prevenzione della corruzione della memoria episodica
- Il coordinamento tra molteplici collaboratori sintetici specializzati
- La manutenibilità a lungo termine delle infrastrutture software

Il codice è l'output. La specifica è la competenza.

Conclusione

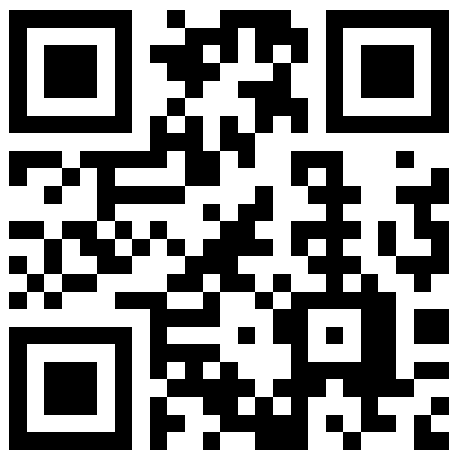
OpenSpec non riguarda lo scrivere meno codice, ma garantire che il codice generato dalle intelligenze artificiali sia **esattamente quello necessario**.

L'arte di definire l'intento è la competenza più preziosa del futuro.

{ Q&A }

Domande? Parliamone.

Contatti



<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



Chi devo ringraziare per queste slide?

- Anthropic: per l'abbonamento Claude Code Max, regalato per i miei contributi al mondo open source
- Codex: per la riformattazione
- Nano Banana Pro: per le immagini
- NotebookLM: per la prima scaletta e i riassunti dei podcast e video
- VSCode: per gestire il progetto GitHub
- Marp: per la presentazione